

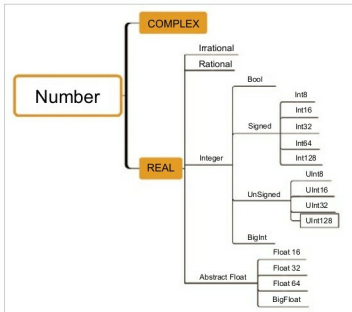


Figure 5: Julia numeric data types

The output is:

```
1332.9215017279294
0.016124 seconds
```

The code was modified to loop through 10 to the power of 8 (100 million) elements, and Julia was able to give the result in 0.120297 seconds.

```
8247.19158291053
0.120297 seconds (2.03 k allocations: 162.361 KB)
```

Note that the successive execution of code is comparatively faster than the first time execution. This is due to the JIT (Just in Time) compilation adopted by Julia.

Let's take a look at another example that involves the multiplication of two matrices of size 1000x1000 and its timing values.

```
a = randn(1000,1000)
b = randn(1000,1000)
@time a*b
```

The output is:

```
0.041690 seconds (9 allocations: 7.630 MB, 5.00% gc time)
```

Julia focuses on numeric processing. It has an extensive hierarchy of types for handling numbers (as shown in Figure 5).

## Type neutral function definition

Julia facilitates the defining of generic functions. Let's consider the following example.

Define a generic function:

```
julia> f(x) = 4x^3+3x^2+8
f (generic function with 1 method)
```

Call f(x) with an integer value:

```
julia> f(5)
583
```

Call f(x) with a float value:

```
julia> f(25.67)
69645.719752
```

Note that we didn't make any modification to the function definition when calling with a different data type. This generic function definition is a very important feature of Julia.

## Cross-language calls

Julia has the option to call functions from languages such as Python, C, Fortran, etc. This facilitates seamless integration between components written in different languages.

For example, to call Python functions from Julia, type:

```
using PyCall
@pyimport math
math.sin(math.pi / 4) - sin(pi / 4) # returns 0.0
```

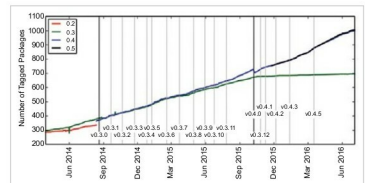


Figure 6: Package ecosystem

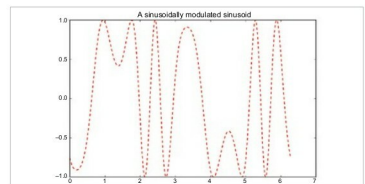


Figure 7: Julia plotting demo with PyPlot

*Continued on page 49...*